



What is Procedure – oriented programming  
Basic Concept of Object-oriented programming paradigm  
Benefits of object-oriented programming  
Application of object-oriented programming input/output operators  
Structure of c++ program  
Introduction of namespace  
Tokens: keywords, identifiers, basic data types  
user-defined types  
derived data types  
symbolic constants  
type compatibility  
declaration of variables  
dynamic initialization of variables, reference variables  
Operators in C++: scope resolution operator  
member referencing operator  
memory management operator  
manipulators  
type cast operator  
Expression: Expression and their types  
Implicit conversions  
Operator precedence

### **Control structures & Looping control structure**

Simple if, if...else, nested if-else, switch etc., for, while, do...while

### **Language Features**

The main function  
Function prototype  
Call by reference  
Return by reference  
Inline function  
Default arguments  
Const arguments  
Functions overloading

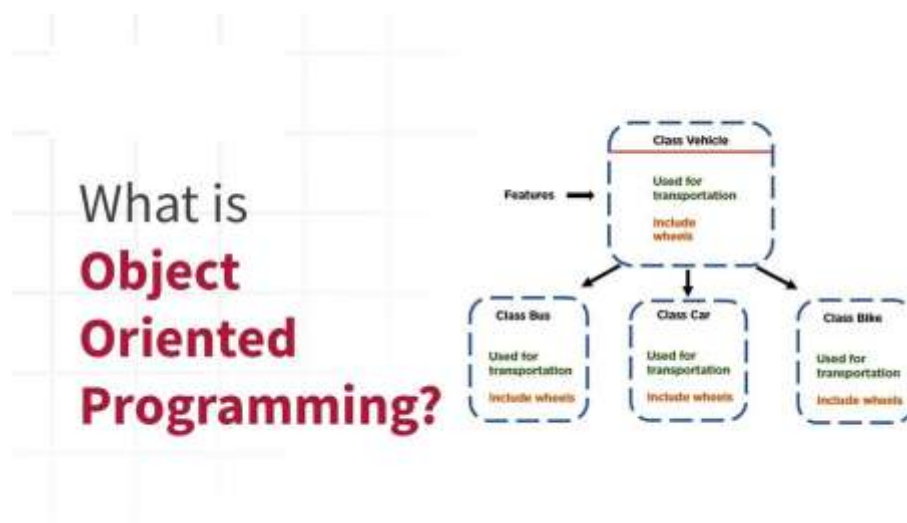
## What is Procedure – oriented programming

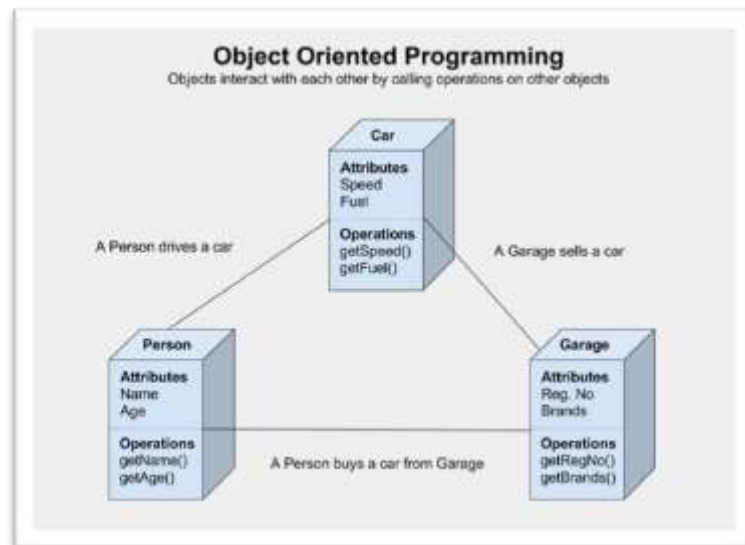
- What is Procedural Programming?
- Procedural programming is a programming language derived from structure programming and based on the concept of calling procedure. The procedures are the functions, routines, or subroutines that consist series of steps to be carried out in the C++ OOPs concept.
- The C++ OOPs concept breaks down a task into a set of variables and routines. After that, it carried out every step one by one. Procedural programming uses a linear top-down approach during the design of the program..

## What Is Object-Oriented Programming (OOP)?

OOP is a programming approach in which data and the functions that work on that data are combined into a single unit called an object.

- As the name suggests, objects in programming. A programming model is based upon the concepts of objects, or where everything is represented as an object. In OOP, we try to see the whole world as an object. For example, dogs have states or data members like color, hungry, and breed, and the behaviors or methods (actions they can perform) are barking, wagging their tail, eating, sleeping, etc.







## **Basic concepts of object-oriented Programming**

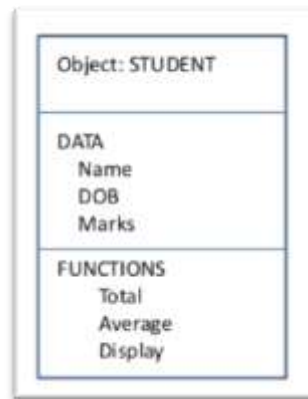
**Objects:** “An Object is an identifiable entity with some characteristics and behavior.”

In OOPs, characteristics are data and behavior is function.

- Objects are the basic run time entities in an object-oriented system.
- They may represent a person, a place, a bank account, a table of data or any item that the program has to handle.
- Programming problem is analyzed in term of objects and the nature of communication between them.
- Program objects should be chosen such that they match closely with the real-world objects.
- 

### **How Object works?**

- When a program is executed, the objects interact by sending messages to one another.
- For example, if “customer” and “account” are object in a program, then the customer object may send a message to the account object requesting for the bank balance.
- Each object contain data, and code to manipulate data (functions).
- Objects can interact without having to know details of each other’s data or code.
- It is a sufficient to know the type of message accepted, and the type of response returned by the objects.



### Classes:

- We just mentioned that objects contain data, and code to manipulate that data.
- The entire set of data and code of an object can be made a user-defined data type with the help of class.
- In fact, objects are variables or instance of the type class.
- Once a class has been defined, we can create any number of objects belonging to that class.
- Each object is associated with the data of type class with which they are created.
- For example, StudentInfo is a Class and Student is an object of StudentInfo Class.

### Encapsulation

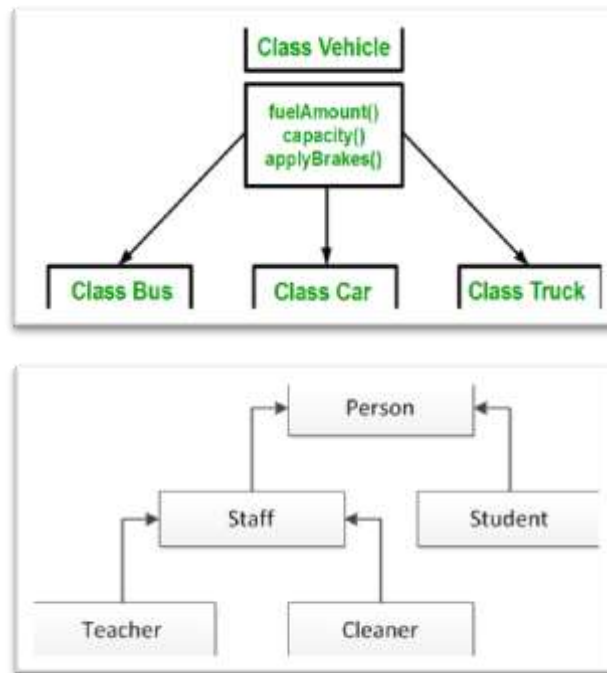
- The wrapping up of data and function into a single unit (called class) is known as encapsulation.
- Data and encapsulation is the most prominent feature of a class.
- The data is not accessible to the outside world, and only those functions which are wrapped in the class can access it.
- Functions provide the interface between the object's data and the program.
- Insulation of the data from direct access by the program is called data hiding or information hiding.

### Abstraction

- Abstraction refers to the act of representing essential features without including the background details or explanation.
- Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight, and cost, and function operate on these attributes.
- The attributes are some time called data members because they hold information. The functions that operate on these data are sometimes called methods or member function.

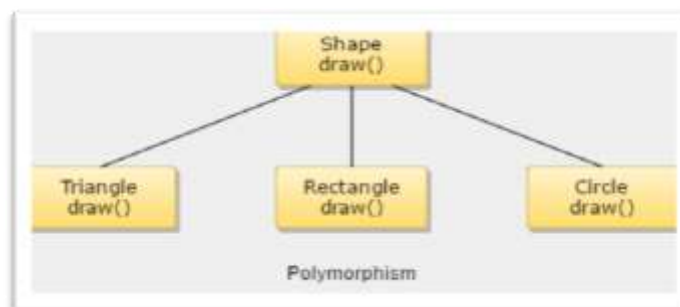
### Inheritance

- Inheritance is the process by which objects of one class acquired the properties of objects of another classes.
- It supports the concept of hierarchical classification.
- For example, class 'Bus' is a part of class 'Vehicle'.
- Each derived class shares common characteristics with the class from which it is derived.
- In OOP, the concept of inheritance provides the idea of reusability.
- The new class will have the combined feature of both the classes.
- Allows the programmer to reuse a class and to tailor the class in such a way that it does not introduced any undesirable side-effects into the rest of classes.



### Polymorphism

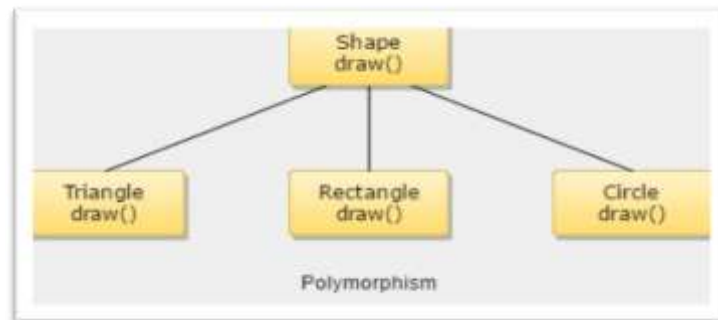
- Polymorphism, a Greek term, means the ability to take more than one form.
- An operation may exhibit different behavior in different instances. The behavior depends upon the types of data used in the operation.
- For example, consider the operation of addition.
- For two numbers, the operation will generate a sum. If the operands are strings, then the operation would produce a third string by concatenation. The process of making an operator to exhibit different behaviors in different instances is known as operator overloading.
- Another example, a single function name can be used to handle different number and different types of argument. Using a single function name to perform different type of task is known as function overloading.



### Dynamic Binding

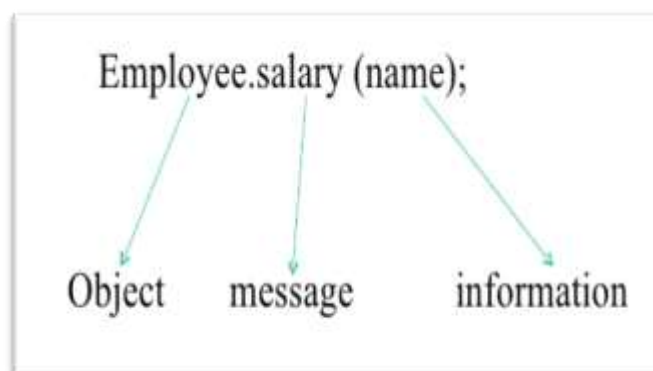
- Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at run time.
- It is associated with polymorphism and inheritance. A function call associated with a polymorphic reference depends on the dynamic type of that reference.
- Consider the procedure “draw” in figure by inheritance, every object will have this procedure.
- Its algorithm is, however, unique to each object and so the draw procedure will be redefined in

each class that defines the object. At run-time, the code matching the object under current reference will be called.



### Message Passing

- An object-oriented program consists of a set of objects that communicate with each other.
- The process of programming in an object-oriented language, involves the following basic steps:
  1. Creating classes that define object and their behavior,
  2. Creating objects from class definitions, and
  3. Establishing communication among objects.
- Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another.
- A Message for an object is a request for execution of a procedure, and therefore will invoke a function (procedure) in the receiving object that generates the desired results.
- Message passing involves specifying the name of object, the name of the function (message) and the information to be sent.



## **Benefits of object-oriented programming**

OOP offers several benefits to both the program designer and the user. The principal advantages are:

- Through inheritance, we can eliminate redundant code extend the use of existing Classes.
- We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
- The principle of data hiding helps the programmer to build secure program that can not be invaded by code in other parts of a programs.
- It is possible to have multiple instances of an object to co-exist without any interference.
- It is easy to partition the work in a project based on objects.
- Object-oriented system can be easily upgraded from small to large system.
- Message passing techniques for communication between objects makes to interface descriptions with external systems much simpler.
- Software complexity can be easily managed.

## **What is C++?**

- C++ is an object-oriented programming language.
- It was developed by Bjarne Stroustrup at AT&T Bell Laboratories in Murray Hill, New Jersey, USA, in the early 1980's.
- C++ is an extension of C with a major addition of the **Class construct feature**.
- Since the Class was a major addition to the original C language, Stroustrup initially called the new language 'C with classes'. However, later in 1983, the name was changed to C++.
- The idea of C++ comes from the C increment operator ++, thereby suggesting that C++ is an augmented version of C.
- C++ is a superset of C.
- The most important facilities that C++ adds on to C are Classes, Inheritance, Function Overloading and Operator Overloading. These features enable creating of abstract data types, inherit properties from existing data types and support polymorphism, thereby making C++ a truly object-oriented language.
- C++ is regarded as a middle-level language, as it comprises a combination of both high-level and low-level language features.
- C++ runs on a variety of platforms, such as Windows, Mac OS, and the various versions of UNIX.

## **Applications of C++**

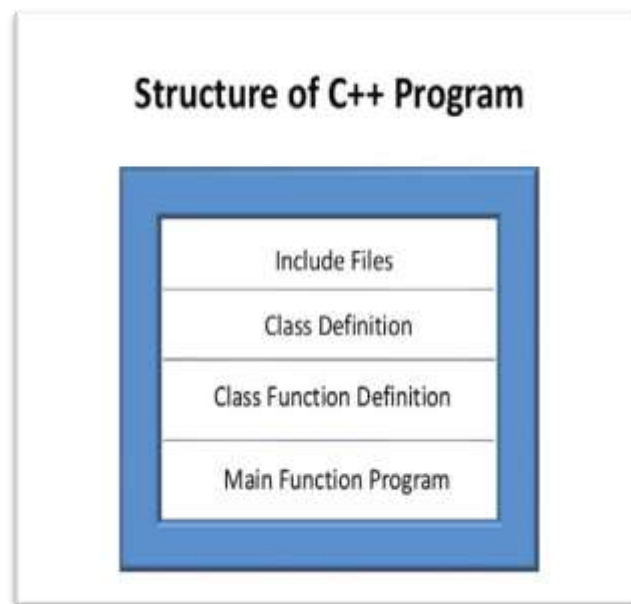
- C++ is a versatile language for handling very large programs; it is suitable for virtually any programming task including development of editors, compilers, databases, communication systems and any complex real life applications systems.
- Since C++ allow us to create hierarchy related objects, we can build special object-oriented libraries which can be used later by many programmers.
- While C++ is able to map the real-world problem properly, the C part of C++ gives the language the ability to get closed to the machine-level details.



- C++ programs are easily maintainable and expandable. When a new feature needs to be implemented, it is very easy to add to the existing structure of an object.
- It is expected that C++ will replace C as a general-purpose language in the near future.

### Structure of C++ Program

- A typical C++ program would contain four sections as shown in figure where each section may be placed in separate code files and then compiled independently or jointly.
- The class declarations are placed in a header file and the definitions of member functions go into another file.
- This approach enables the programmer to separate the abstract specification of the interface from the implementation details (member function definition).
- Finally, the main program that uses the class is placed in a third file which “includes: the previous two files as well as any other file required.



- **// comment**
  - C++ introduces a new comment symbol // (double slash). Comment start with a double slash symbol and terminate at the end of the line. The double slash comment is basically a single line comment.
- **#include <iostream>**
  - The #include directive instructs the compiler to include the contents of the file enclosed within angular brackets into the source file. The header file iostream.h should be included at the beginning of all programs that use input/output statements.
- **using namespace std**
  - Namespace is a new concept introduced by the ANSI C++ standards committee. This defines a scope for the identifiers that are used in a program. Here, std is the namespace where ANSI C++ standard class libraries are defined. All ANSI C++ programs must include this directive.

Program1.cpp

```
//simple c++ program
#include <iostream>
using namespace std;
void main() {
    int a;
    cout << "Hello World";
    cin >> a;
    cout << a;
}
```

- **void main()**
  - The main() function is the entry point of every program in C++ language. The void keyword specifies that it returns no value.
- **Cout statement**
  - Causes the string in quotation marks to be displayed on the screen. This statement introduces two new C++ features, cout and <<. The identifier cout(pronounced as C out) is a predefined object that represents the standard output stream in C++.
- **Cin statement**
  - causes the program to wait for the user to type in a number. The number keyed in is placed in the variable number1. The identifier cin (pronounced 'C in') is a predefined object in C++ that corresponds to the standard input stream.

### Compile and run C++ Program

#### Turbo C++

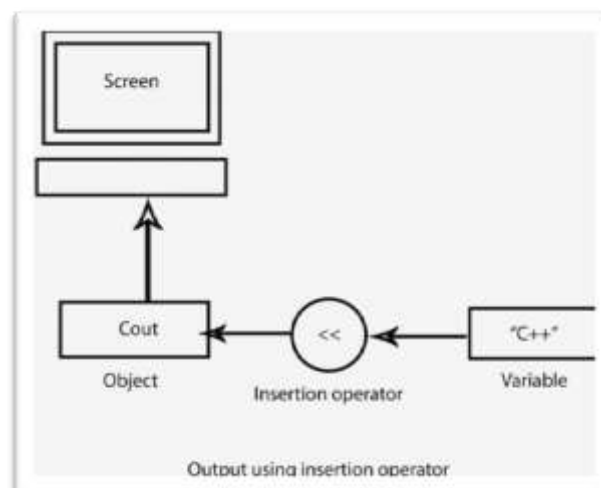
- Turbo C++ provides an integrated program development environment under MS DOS. They provide a built-in editor and a menu bar includes options such as File, Edit, Compile and Run.
- We can create and save the source files under the File option, and edit them under the Edit option.
- We can then compile the program under the compile option and execute it under the Run option.

## Application of object-oriented programming input/output operators

### The Standard Output Stream (cout)

- The predefined object cout is an instance of ostream class. The cout object is said to be "connected to" the standard output device, which usually is the display screen. The cout is used in conjunction with the stream insertion operator, which is written as << which are two less than signs as shown in the following example.
- When the above code is compiled and executed, it produces the result – Value of str is : Hello C++
- The C++ compiler also determines the data type of variable to be output and selects the appropriate stream insertion operator to display the value. The << operator is overloaded to output data items of built-in types integer, float, double, strings and pointer values.
- The insertion operator << may be used more than once in a single statement as shown above and endl is used to add a new-line at the end of the line.

```
#include <iostream>
using namespace std;
int main() {
    char str[] = "Hello C++";
    cout << "Value of str is" << str << endl;
}
Output:
Value of str is : Hello C++
```



## The Standard Input Stream (cin)

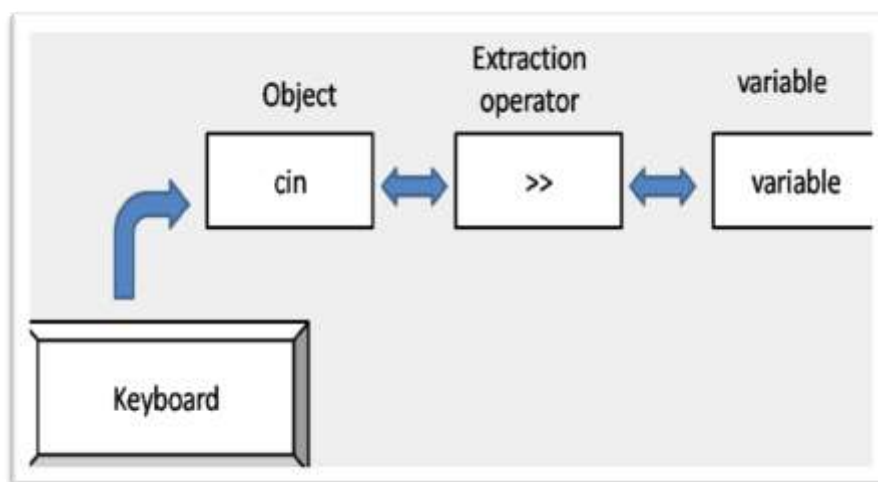
- The predefined object cin is an instance of istream class. The cin object is said to be attached to the standard input device, which usually is the keyboard. The cin is used in conjunction with the stream extraction operator, which is written as >> which are two greater than signs as shown in the example.
- When the code is compiled and executed, it will prompt you to enter a name. You enter a value and then hit enter to see the result.
- The stream extraction operator >> may be used more than once in a single statement. To request more than one datum you can use the following –
- `cin >> name >> age;`

```
#include <iostream>
using namespace std;
int main()
{
    char name[50];
    cout << "Please enter your name: ";
    cin >> name;
    cout << "Your name is: " << name << endl;
}
```

Output:

Please enter your name: ABC

Your name is: ABC



### Token

- A token is the smallest element of a program that is meaningful to the compiler. Tokens can be classified as follows:
  - Keywords
  - Identifiers
  - Constants
  - Strings
  - Special Symbols
  - Operators

### Keyword

- Keywords are pre-defined or reserved words in a programming language.
- Each keyword is meant to perform a specific function in a program.
- Since keywords are referred names for a compiler, they can't be used as variable names
- You cannot redefine keywords.
- In C++ there are 31 additional keywords other than C Keywords which are as below:

|             |           |              |                  |
|-------------|-----------|--------------|------------------|
| asm         | bool      | catch        | class            |
| const_cast  | delete    | dynamic_cast | explicit         |
| export      | false     | friend       | inline           |
| mutable     | namespace | new          | operator         |
| private     | protected | public       | reinterpret_cast |
| static_cast | template  | this         | throw            |
| true        | try       | typeid       | typename         |
| using       | virtual   | wchar_t      |                  |

### Identifier

- Identifiers are used as the general terminology for naming of variables, functions and arrays.
- Identifier names must differ in spelling and case from any keywords.
- You cannot use keywords as identifiers.
- Once you declared identifier, you can use the identifier in later program statements to refer to the associated value.

There are certain rules that should be followed while naming c identifiers:

1. They must begin with a letter or underscore(\_).
2. They must consist of only letters, digits, or underscore. No other special character is allowed.
3. It should not be a keyword.
4. It must not contain white space.
5. It should be up to 31 characters long as only first 31 characters are significant.

| NAME     | REMARK                                                             |
|----------|--------------------------------------------------------------------|
| _A9      | Valid                                                              |
| Temp.var | Invalid as it contains special character other than the underscore |
| void     | Invalid as it is a keyword                                         |

### Constants

- Constants are also like normal variables. But, only difference is, their values can not be modified by the program once they are defined.
- Constants refer to fixed values. They are also called as literals.
- Constants may belong to any of the data type.

Types of Constants:

- Integer constants – Example: 0, 1, 1218, 12482
- Real or Floating point constants – Example: 0.0, 1203.03, 30486.184
- Octal & Hexadecimal constants – Example: octal:  $(013)_8 = (11)_{10}$ , Hexadecimal:  $(013)_{16} = (19)_{10}$
- Character constants -Example: 'a', 'A', 'z'
- String constants -Example: "HelloWorld".

### **Syntax:**

```
const data_type variable_name;  
(or)  
const data_type *variable_name;
```

## Strings

- Strings are nothing but an array of characters ended with a null character ('\\0').
- This null character indicates the end of the string.
- Strings are always enclosed in double quotes.
- Declarations for String:
  - `char string[20] = {'H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd', '\\0'};`
  - `char string[20] = "HelloWorld";`
  - `char string [] = "HelloWorld";`

Difference between above declarations are:

- when we declare char as "string[20]", 20 bytes of memory space is allocated for holding the string value.
- When we declare char as "string[]", memory space will be allocated as per the requirement during execution of the program.

## Special Symbols

- The following special symbols are used for some special meaning and thus, cannot be used for some other purpose.

`[] () {} , ; * = #`

- **Brackets[]:**
  - Opening and closing brackets are used as array element reference. These indicate single and multidimensional subscripts.
- **Parentheses():**
  - These special symbols are used to indicate function calls and function parameters.
- **Braces{}:**
  - These opening and ending curly braces marks the start and end of a block of code containing more than one executable statement.
- **comma (,):**
  - It is used to separate more than one statements like for separating parameters in function calls.
- **semi colon :**
  - It is an operator that essentially invokes something called an initialization list.
- **asterick (\*):**

- It is used to create pointer variable.
- **assignment operator:**
  - It is used to assign values.
- **pre processor(#):**
  - The preprocessor is a macro processor that is used automatically by the compiler to transform your program before actual compilation.

## **Operators**

Operators are symbols that triggers an action when applied to C/C++ variables and other objects. The data items on which operators act upon are called operands. Operators can be classified as follows:

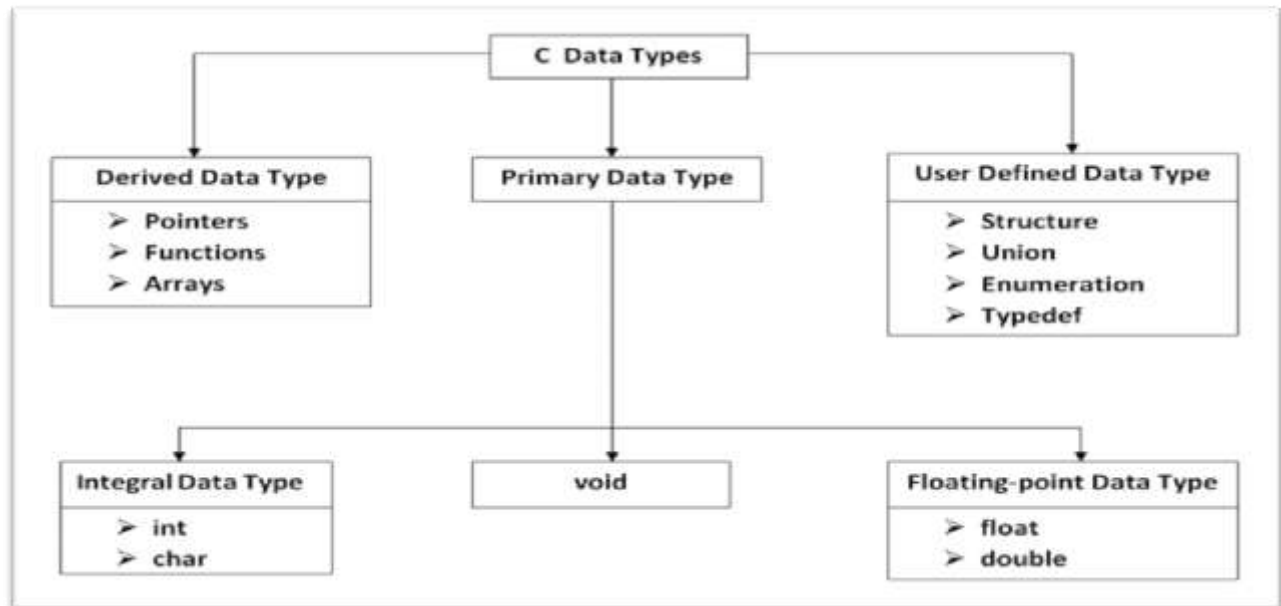
- **Unary Operators:** Those operators that require only single operand to act upon are known as unary operators .For Example increment and decrement operators
- **Binary Operators:** Those operators that require two operands to act upon are called binary operators. Binary operators are classified into :
  - Arithmetic operators (+,-,\*,/,%)
  - Relational Operators (<,>,<=,>=,==,!=)
  - Logical Operators ( &&, ||, !)
  - Assignment Operators (=, +=, -=, \*=, /=, %=)
  - Bitwise Operators ( &, |, ^, <<, >>, ~)
- **Ternary (Conditional) Operators:** These operators requires three operands to act upon. For Example Conditional operator(?:).



## Data types in C++

- A data type specifies the type of data that a variable can store such as integer, floating, character etc.

In C++, it is basically divided into 3 types.



- The basic (fundamental) data types provided by c++ are **integral, floating point and void** data type.
- **Integral** type : – The data types in this type are **int and char**.
  - The modifiers signed, unsigned, long & short may be applied to character & integer basic data type.
  - The size of int is 2 bytes and char is 1 byte.
- **Floating** type: The data types in this are **float & double**.
  - The size of the float is 4 byte and double is 8 byte.
  - The modifier long can be applied to double & the size of long double is 10 byte.
- **Void** – void is used :
  - To specify the return type of a function when it is not returning any value.
  - To indicate an empty argument list to a function.
    - Ex- void function1(void)
  - In the declaration of generic pointers.
    - Ex- void \*gp

- A generic pointer can be assigned a pointer value of any basic data type.
  - Ex . `int *ip` // this is int pointer
  - `gp = ip` //assign int pointer to void.
- A void pointer cannot be directly assigned to their type pointers in c++ we need to use cast operator.
  - Ex – `void *ptr1;`
  - `char *ptr2;`
  - `ptr2 = ptr1;` is allowed in c but not in c++.
  - `ptr2 = (char *)ptr1;` is the correct statement.

## User-Defined Data Types

### Structure, Union, Classes, Enumeration

- The user-defined data type **Structure and Union** are same as that of C.
- Classes – Class is a user defined data type which can be used just like any other basic data type once declared. The class variables are known as objects.
- Enumeration - An enumerated data type is another user defined type which provides the way of attaching names to numbers to increase simplicity of the code.
  - It uses **enum** keyword which automatically enumerates a list of words by assigning them values 0, 1, 2,.....etc.

Syntax:- **enum shape { circle, square, triangle; }**

Now shape becomes a new type name & we can declare new variables of this type. Ex . `shape oval;`

- In C++, enumerated data type has its own separate type. Therefore C++ does not permit an int value to be automatically converted to an enum value.
  - `shape shapes1 = triangle,` // is allowed shape
  - `shape1 = 2;` // Error in C++
  - `shape shape1 = (shape)2;` //ok

By default, enumerators are assigned integer values starting with 0, but we can override the default value by assigning some other value.

```
enum colour {red, blue, pink = 3};
```

//it will assign red to 0, blue to 1, & pink to 3 or

```
enum colour {red = 5, blue, green};
```

//it will assign red to 5, blue to 6 & green to 7

## **Derived Data types**

### ▪ **Arrays**

- An array is collection of similar datatype elements. An array in c++ is similar to that in c,
- The only difference is the way character arrays are initialized. In c++, the size should be one larger than the number of character in the string where in c, it is exact same as the length of string constant. Ex –

- `char string1[3] = "ab"; // in c++`

- `char string1[2] = "ab"; // in c`

### ▪ **Functions**

- Functions in c++ are different than in c there is lots of modification in functions in c++ due to object orientated concepts in c++ that we will discuss later in detail.

### ▪ **Pointers**

- Pointers are declared & initialized as in c.
  - `int * ip; // int pointer`
  - `ip = &x; //address of x through indirection`
- c++ adds the concept of constant pointer & pointer to a constant pointer.
  - `char const *p2 = "HELLO" // constant pointer`

## **Variable Declaration and Initialization**

- Variable are used in C++, where we need storage for any value, which will change in program.
- Variable can be declared in multiple ways each with different memory requirements and functioning.
- Variable is the name of memory location allocated by the compiler depending upon the datatype of the variable.

### Declaration and Initialization

- Variable must be declared before they are used. Usually it is preferred to declare them at the starting of the program, but in C++ they can be declared in the middle of program too, but must be done before using them.
- For example,
  - `int i; // declared but not initialized`
  - `char c; int i, j, k; // Multiple declaration`
- Initialization means assigning value to an already declared variable,

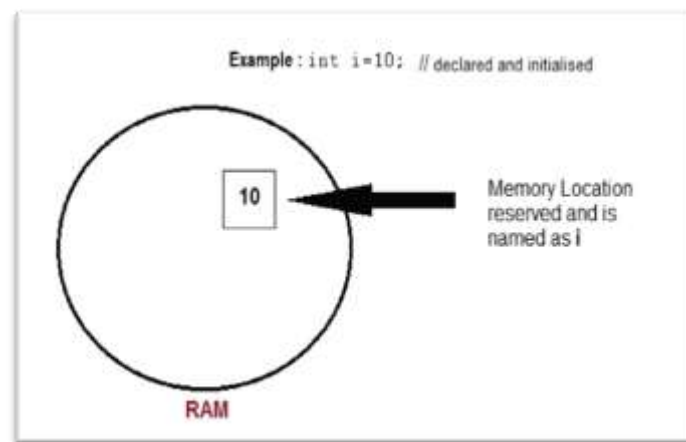
- `int i; // declaration`
- `i = 10; // initialization`
- If a variable is declared and not initialized by default it will hold a garbage value. Also, if a variable is once declared and if try to declare it again, we will get a compile time error.

```
int i,j;
```

```
i=10;
```

```
j=20;
```

```
int j=i+j; //compile time error, cannot redeclare a variable in same scope.
```



### Example of Variable Declaration & Initialization

```
#include <iostream>

using namespace std;

int main() {
    char a;
    int b=30;
    a= 's';
    cout << "\nThe variable a = " << a;
    cout << "\nThe variable b = " << b;
    return 0;
}
```

Output:  
The variable a = s  
The variable b = 30

## Dynamic Initialization Of Variables

- **The process of initializing variables at run time is called dynamic initialization.**
- This feature is supported by C++ and not C.
- In C++, a variable can be initialized at run time **using expressions at the place declaration**
- Both declaration and initialization of a variable can be done simultaneously at the place using expression where the variable is used for the first time. Example
  - `int n=strlen(string);`
  - `float area=3.1459 * rad * rad;`
- For example, following statements can be **merged** into a single statement which initialize the average variable **dynamically at run time**.
  - `Float average;`
  - `Average=sum/I;`
- This can be combined into a single statement
  - `Float average=sum/I;`

// initialize dynamically at run time.

- **Dynamic initialization is extensively used in object oriented programming**

## Reference variables

- It is a new variable introduced in C++.
- A reference variable provides an alias (alternative name) for a previously defined variable.
- E.g., if we make the variable sum a reference variable to the variable total, then sum and total can be used interchangeably to represent that variable.

Syntax

- `Data type & reference_ name=variable_name;`

Example

```
Float total=100;
```

```
Float & sum=total;
```

- Total is a float type variable that has already been declared;
- sum is an alternative name declared to represent the variable total.



- Both the variables refer to the same data object in the memory.
- The statement
  - `cout<<total; and`
  - `cout<<sum;`
  - both will print the value 100.
- The statement `total=total +10;` will change the value of both total and sum to 110.
- Likewise, the assignment `sum=0;` will change the value of both variables to zero.
- A reference variable must be initialized at the time of declaration.
- C++ assigns additional meaning to the symbol `&`.

`&` is not an address operator, therefore the notation `float &` means reference to float.

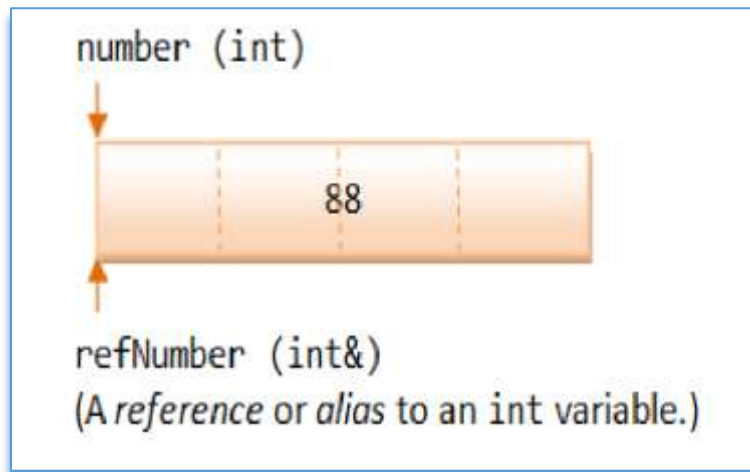
### Example of Reference Variable in C++

```
#include<iostream>

using namespace std;

int main()
{
    int number = 10;
    int& refNumber = number; // refNumber is a reference to number.
    refNumber = 20;         // Value of number is now changed to 20
    cout << " number = " << number << endl ;
    number = 30; // Value of number is now changed to 30
    cout << "refNumber = " << refNumber << endl ;
    return 0;
}
```

Output:  
number = 20  
refNumber = 30



## Operators in C++

- **Scope Resolution Operator**
- **Member Referencing Operator**
- **Memory Management Operator**
- **Manipulators**
- **Type Cast Operator**

### **Scope resolution operator**

- The scope resolution operator ( :: ) is used for several reasons.
- For example:
  - If the global variable name is same as local variable name, the scope resolution operator will be used to call the global variable.
  - It is also used to define a function outside the class and used to access the static variables of class.

## Example of Scope Resolution Operator

```
#include <iostream>

using namespace std;

char a = 'm';
int b = 10;

int main() {
    char a = 's';
    int b=30;
    cout << "\nThe local variable a = " << a;
    cout << "\nThe global variable ::a = " << ::a;
    cout << "\nThe local variable b = " << b;
    cout << "\nThe global variable ::b = " << ::b;
    return 0;
}
```

Output:

The local  
variable a = s

The global

## Member Referencing Operator

- C++ permits us to define a class containing various types of data and functions as members.
- C++ also permits us to access the class members through pointers.
- In order to achieve this, C++ provides a set of three pointer-to-member operators.
- Note: We will discuss these operators in detail after learning Concept of Classes.

| Operator | Function                                                                      |
|----------|-------------------------------------------------------------------------------|
| ::*      | To declare a pointer to a member of a class                                   |
| *        | To access a member using object name and a pointer to that member             |
| →*       | To access a member using a pointer to the object and a pointer to that member |

- c uses malloc() and calloc() functions to allocate memory dynamically at run time,
- Similarly, it uses the function free() to free dynamically allocated memory.
- C++ support these functions.
- It also defines two unary operators new and delete that perform the task of allocating and freeing the memory in a better and easier way.



- Since these operators manipulate memory on the free store, they are also known as **free store operators**.

- The **new** operator can be used to create objects of any type.

- It takes the following general form:

- **pointer – variable = new data – type(value);**

- EXAMPLES:

```
p = new int;
```

```
q = new float;
```

- Alternatively, We can combine the declaration of pointers and their assignment as follows:

```
int *p = new int;
```

```
float *q = new float;
```

- Subsequently ,the statements

```
*p = 25;
```

```
*q = 7.5;
```

- assign 25 to newly created int object and 7.5 to newly created float object.

- When a data object is no longer needed, it is destroyed to release the memory space for reuse using **delete** operator.

- The general form of its use is:

**delete pointer – variable;**

- EXAMPLES:

```
delete p;
```

```
delete q;
```

- The new operator offers the following advantages over the function malloc().

- It automatically computes the size of the sate object. We need not use the operator sizeof.
- It automatically returns the correct pointer type, so that there is no need to use a type cast.
- It is possible to initialize the object while creating the memory space.
- Like any other operator, new and delete can be overloaded.

## **Example Of Memory Management Operators**

```
#include <iostream>

using namespace std;

int main () {
    int *ptr1 = NULL;
    ptr1 = new int;
    float *ptr2 = new float(223.324);
    int *ptr3 = new int[28];
    *ptr1 = 50;
    cout << "Value of pointer variable ptr1 : " << *ptr1 << endl;
    cout << "Value of pointer variable ptr2 : " << *ptr2 << endl;
    if (!ptr3)
    {
        cout << "Allocation of memory failed\n";
    }
    else
    {
        for (int i = 0; i < 28; i++)
        {
            ptr3[i] = i+1;
        }
        cout << "Value of store in block of memory: ";
        for (int i = 0; i < 28; i++)
        {
            cout << ptr3[i] << " ";
        }
    }
    delete ptr1;
    delete ptr2;
```

```
delete[] ptr3;  
  
// cout<< *ptr1; it will show garbage value of ptr1  
  
return 0;  
  
}
```

#### Output

Value of pointer variable ptr1 : 50

Value of pointer variable ptr2 : 223.324

Value of store in block of memory: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19  
20 21 22 23 24 25 26 27 28

### Manipulators

- Manipulators are operators that are used to format the data display.
- The most commonly used manipulators are **endl** and **setw**.
- The **endl** manipulator, when used in an output statement, causes a linefeed to be inserted. It has the same effect as using the newline character “\n”.

- EXAMPLES:

```
cout<<“m = “ << m << endl << “n = “ << n << endl << “p = “ << p << endl;
```

- This form of the output is possible only if we can specify a common field width for all the numbers and force them to be printed right-justified.
- The **setw** manipulator does this job. It is used as follows:

```
cout << setw(5) << sum << endl;
```

### Example Of Manipulators

```
// setw example  
  
#include <iostream>  
  
#include <iomanip>    // std::setw  
  
using namespace std;  
  
int main () {  
    int a=10;  
    int b=1000;  
    int c=100;
```

```
cout << "a: " << setw(5) << a << endl;  
cout << "b: " << setw(5) << b << endl;  
cout << "c: " << setw(5) << c << endl;  
return 0;  
}
```

Output:

```
a:   10  
b: 1000  
c:  100
```

### Type Cast Operator

- C++ permits explicit type conversion of variables or expressions using the type cast operator.
- Traditional c casts are augmented in C++ by a function call notation as a syntactic alternative.
- The following two versions are equivalent:
  - (type – name) expression // C notation
  - type – name (expression) // C++ notation
- EXAMPLES:
  - average = sum/(float) i; // C notation
  - average = sum/float (i); // C++ notation

### Example Of Type Cast Operator

```
#include <iostream>  
using namespace std;  
int main () {  
    int a=10;  
    int b=7;  
    float ans = a/b;  
    cout << "Result of a/b: " << ans << endl;  
    ans = float(a)/b; //ans = (float)a/b  
    cout << "Result of a/b: " << ans << endl;  
    return 0;  
}
```

Output

Result of a/b: 1

Result of a/b: 1.42857

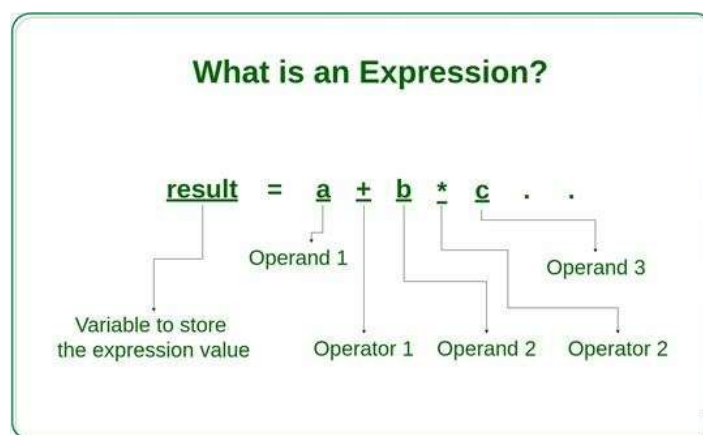
## Expression

- C++ expression consists of operators, constants, and variables which are arranged according to the rules of the language.
- It can also contain function calls which return values. An expression can consist of one or more operands, zero or more operators to compute a value.
- Every expression produces some value which is assigned to the variable with the help of an assignment operator.

Examples of C++ expression:

- $(a+b) - c$
- $(x/y) - z$
- $4a^2 - 5b + c$

$(a+b) * (x+y)$



An expression can be of following types:

1. Constant expressions
2. Integral expressions
3. Float expressions
4. Pointer expressions
5. Relational expressions
6. Logical expressions
7. Bitwise expressions
8. Special assignment expressions

If the expression is a **combination of the above expressions**, such expressions are known as **compound expressions**

## Constant expressions

- A constant expression is an expression that **consists of only constant values.**
- It is an expression whose **value is determined at the compile-time but evaluated at the run-time.**
- It can be composed of integer, character, floating-point, and enumeration constants.
- Constants are used in the following situations:
  - It is used in the subscript declarator to describe the array bound.
  - It is used after the case keyword in the switch statement.
  - It is used as a numeric value in an **enum**
  - It specifies a bit-field width.
  - It is used in the pre-processor **#if**

| Expression containing constant | Constant value |
|--------------------------------|----------------|
| $x = (2/3) * 4$                | $(2/3) * 4$    |
| <code>extern int y = 67</code> | 67             |
| <code>int z = 43</code>        | 43             |
| <code>static int a = 56</code> | 56             |

## Constant expressions – Example

```
#include <iostream>
using namespace std;
int main()
{
    int x;    // variable declaration.
    x=(3/2) + 2; // constant expression
    cout<<"Value of x is : "<<x; // displaying the value of x.
    return 0;
}
```

- In the above code, we have first declared the 'x' variable of integer type. After declaration, we assign the simple constant expression to the 'x' variable.



Output

Value of x is : 3

### Integral Expressions

- An integer expression is an expression that **produces the integer value as output** after performing all the explicit and implicit conversions.
- Following are the examples of integral expression:
  - $(x * y) - 5$
  - $x + \text{int}(9.0)$
- where x and y are the integers.

### Integral expressions - Example

```
#include <iostream>
using namespace std;
int main()
{
    int x, y, z; // variable declaration.
    cout<<"Enter the values of x and y";
    cin>>x>>y;
    z=x+y;
    cout<<"\n"<<"Value of z is :"<<z; // displaying the value of z.
    return 0;
}
```

- In the above code, we have declared three variables, i.e., x, y, and z.
- After declaration, we take the user input for the values of 'x' and 'y'.
- Then, we add the values of 'x' and 'y' and stores their result in 'z' variable.

### Output

Enter the values of x and y

8

9

Value of z is :17

## Pointer Expressions

- A pointer expression is an expression that produces address value as an output.
- The following are the examples of pointer expression:

&x

ptr

ptr++

ptr-

## Pointer expressions – Example

```
#include <iostream>
using namespace std;
int main()
{
    int a[]={1,2,3,4,5}; // array initialization
    int *ptr;    // pointer declaration
    ptr=a;    // assigning base address of array to the pointer ptr
    ptr=ptr+1; // incrementing the value of pointer
    cout <<"value of second element of an array : " << *ptr<<endl;
    return 0;
}
```

- In the above code, we declare the array and a pointer ptr. We assign the base address to the variable 'ptr'.
- After assigning the address, we increment the value of pointer 'ptr'. When pointer is incremented then 'ptr' will be pointing to the second element of the array.

## Output

value of second element of an array : 2

## Relational Expressions

- A relational expression is an expression that produces a value of type bool, which can be either true or false.



- It is also known as a **boolean expression**.
- When arithmetic expressions are used on both sides of the relational operator, arithmetic expressions are evaluated first, and then their results are compared.
- The following are the examples of the relational expression:

a>b

a-b >= x-y

a+b>80

### Relational expressions – Example

```
#include <iostream>

using namespace std;

int main()
{
    int a=45; // variable declaration
    int b=78; // variable declaration
    bool y= a>b; // relational expression
    cout<<"Value of y is :"<<y; // displaying the value of y.
    return 0;
}
```

- In the above code, we have declared two variables, i.e., 'a' and 'b'.
- After declaration, we have applied the relational operator between the variables to check whether 'a' is greater than 'b' or not.

### Output

Value of y is :0

### Logical Expressions

- A logical expression is an expression that **combines two or more relational expressions** and produces **a bool type value**.
- The logical operators are **'&&' and '||'** that combines two or more relational expressions.
- The following are some examples of logical expressions:

`a>b && x>y`

`a>10 || b==5`

### LOGICAL expressions - Example

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a=2;
```

```
int b=7;
```

```
int c=4;
```

```
cout<<((a>b)||(a>c));
```

```
return 0;
```

```
}
```

- In the above code, we have declared three variables, i.e., 'a' , 'b' and 'c'.
- After declaration, we have applied the relational operator between the variables and combined two expressions using logical operator to check whether 'a' is greater than 'b' and 'c' or not.

### Output

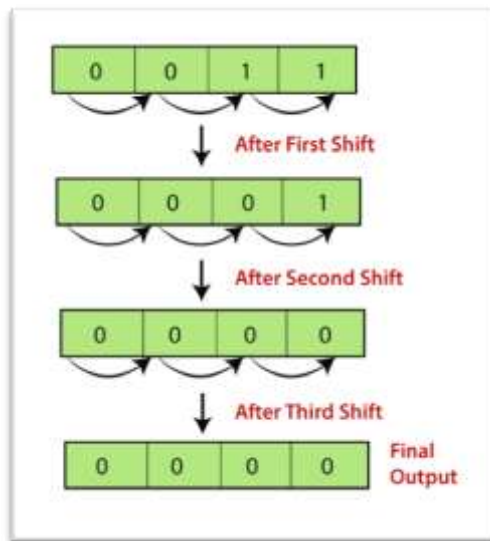
0

### Bitwise Expressions

- A bitwise expression is an expression which is used to manipulate the data at a bit level. They are basically used to shift the bits.
- For example:

`x=3`

`x>>3` // This statement means that we are shifting the three-bit position to the right



### BITWISE expressions – Example

```
#include <iostream>

using namespace std;

int main()
{
    int x=5; // variable declaration
    cout << (x>>1) << endl;
    return 0;
}
```

- In the above code, we have declared a variable 'x'.
- After declaration, we applied the bitwise operator, i.e., right shift operator to shift one-bit position to right.

### Output

2

### Special Assignment Expressions

- Special assignment expressions are the expressions which can be further classified depending upon the value assigned to the variable.

### Chained Assignment

- Chained assignment expression is an expression in which the same value is assigned to more than one variable by using single statement.
- For example:



- `a=b=20`
- `or`
- `(a=b) = 20`

### Example of Chained Assignment

```
#include <iostream>

using namespace std;

int main() {
    int a; // variable declaration
    int b; // variable declaration
    a=b=80; // chained assignment
    cout <<"Values of 'a' and 'b' are : " <<a<<" "<<b;
    return 0;
}
```

In the above code, we have declared two variables, i.e., 'a' and 'b'. Then, we have assigned the same value to both the variables using chained assignment expression.

### Output

Values of 'a' and 'b' are : 80,80

### Special Assignment Expressions

#### Embedded Assignment Expression

An embedded assignment expression is an assignment expression in which assignment expression is enclosed within another assignment expression.

```
#include <iostream>

using namespace std;

int main()
{
    int a; // variable declaration
    int b; // variable declaration
    a=10+(b=90); // embedded assignment expression
    cout <<"Values of 'a' is " <<a<< endl;
}
```

```
return 0;
```

```
}
```

In the above code, we have declared two variables, i.e., 'a' and 'b'. Then, we applied embedded assignment expression ( $a=10+(b=90)$ ).

### Output

Values of 'a' is 100

### Compound Assignment

A compound assignment expression is an expression which is a combination of an assignment operator and binary operator.

For example,

```
a += 10;
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int a=10; // variable declaration
```

```
a+=10; // compound assignment
```

```
cout << "Value of a is :" <<a<< endl;
```

```
// displaying the value of a.
```

```
return 0;
```

```
}
```

In the above code, we have declared a variable 'a' and assigns 10 value to this variable. Then, we applied compound assignment operator (+) to 'a' variable, i.e.,  $a+=10$  which is equal to ( $a=a+10$ ). This statement increments the value of 'a' by 10.

### Output

Value of a is :20

### Implicit Conversion

- The type conversion that is done automatically done by the compiler is known as implicit type conversion. This type of conversion is also known as automatic conversion.
- In the program, we have assigned an int data to a double variable.
- `num_double = num_int;`

- Here, the int value is automatically converted to double by the compiler before it is assigned to the num\_double variable.

// Working of implicit type-conversion int to double

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int num_int = 9; // assigning an int value to num_int
```

```
    double num_double; // declaring a double type variable
```

**//implicit conversion → assigning int value to a double variable**

```
    num_double = num_int;
```

```
    cout << "num_int = " << num_int << endl;
```

```
    cout << "num_double = " << num_double << endl;
```

```
    return 0;
```

```
}
```

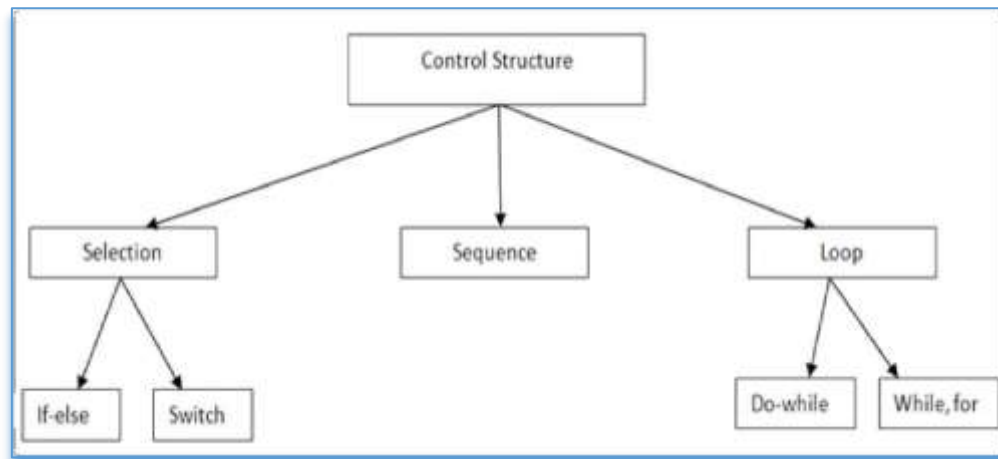
### Output

```
num_int = 9
```

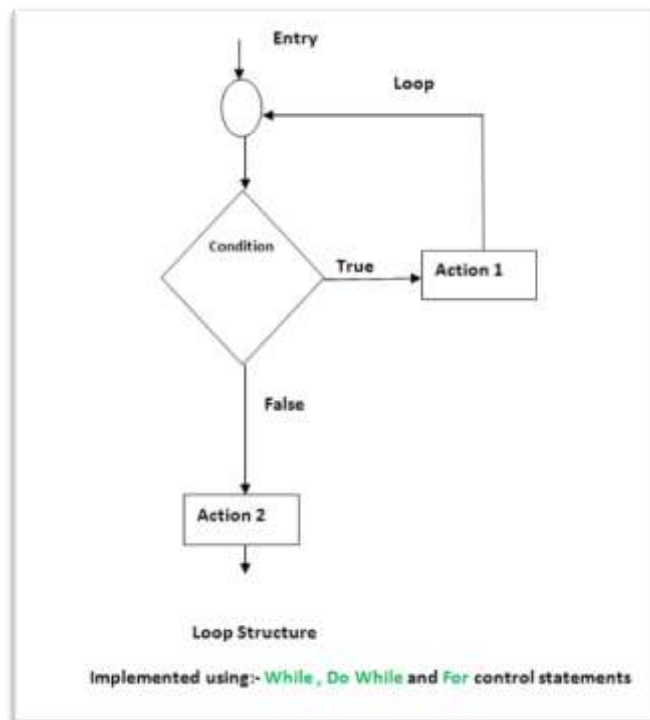
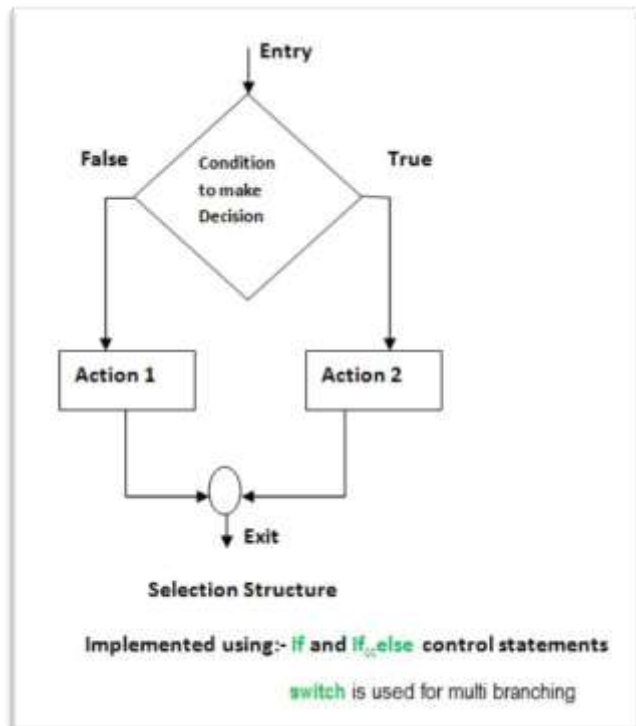
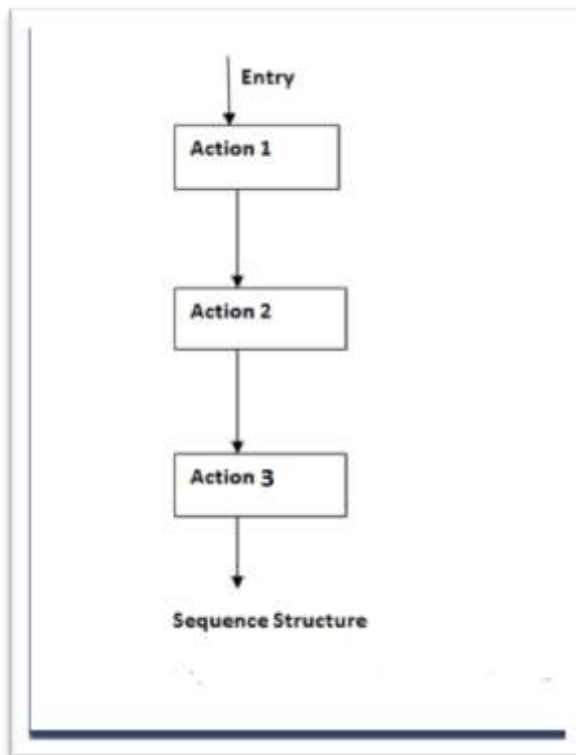
```
num_double = 9
```

### Control Structures

- Control structures are used to alter the flow of execution of the program.
- We use control structures to make decisions and alter the direction of program flow in one or the other path(s).
- There are three types of control structures available in C and C++
  - 1) Sequence structure (straight line paths)
  - 2) Selection structure (one or many branches)
  - 3) Loop structure (repetition of a set of activities)



### Basic Control Structures



### Selection Structure / Decision Making Structure

- Decision making is about deciding the order of execution of statements based on certain conditions
- C++ handles decision-making by supporting the following statements,





- *if* statement
- *switch* statement
- conditional operator statement
- *goto* statement

### Decision making with if statement

- The if statement may be implemented in different forms depending on the complexity of conditions to be tested. The different forms are,
  - Simple if statement
  - if...else statement
  - Nested if...else statement
  - else if statement

### If statement

- The C++ if statement tests the condition. It is executed if condition is true.
- Syntax

```
if(condition){  
    //code  
}  
  
#include <iostream>  
using namespace std;  
int main () {  
    int num = 10;  
    if (num % 2 == 0)  
    {  
        cout<<"It is even number";  
    }  
    return 0;  
}
```

### Output:

It is even number



### If...else statement

- The C++ if-else statement also tests the condition.
- It executes “if block” if condition is true otherwise “else block” is executed.
- Syntax

```
if(condition){  
    //code if condition is true  
}  
else{  
    //code if condition is false  
}  
  
#include <iostream>  
using namespace std;  
  
int main () {  
    int num;  
    cout<<"Enter a Number: ";  
    cin>>num;  
    if (num % 2 == 0)  
    {  
        cout<<"It is even number"<<endl;  
    }  
    else  
    {  
        cout<<"It is odd number"<<endl;  
    }  
    return 0;  
}
```

### Output:

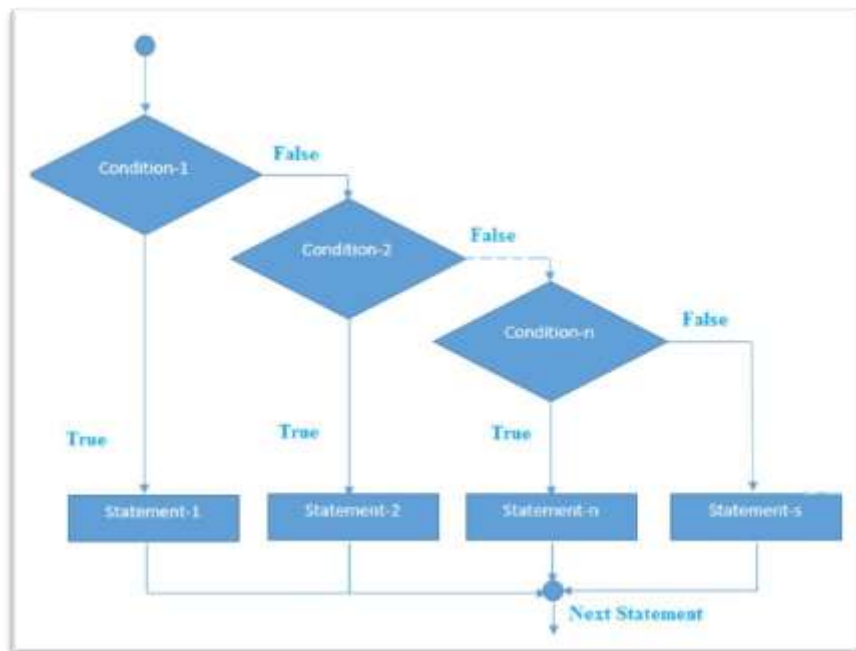
Enter a number:11

It is odd number

### If...Else-if statement

- The C++ if-else-if ladder statement executes one condition from multiple statements.

```
if(condition1){  
    //code to be executed if condition1 is true  
}  
else if(condition2){  
    //code to be executed if condition2 is true  
}  
else if(condition3){  
    //code to be executed if condition3 is true  
}  
...  
else{  
    //code to be executed if all the conditions are false  
}
```



### If...else-if statement - Example

```
#include <iostream>

using namespace std;

int main () {
    int num;

    cout<<"Enter a number to check grade:";

    cin>>num;

    if (num <0 || num >100)
    {
        cout<<"wrong number";
    }
    else if(num >= 0 && num < 50){
        cout<<"Fail";
    }

    else if (num >= 50 && num < 60)
    {
        cout<<"D Grade";
    }

    else if (num >= 60 && num < 70)
    {
        cout<<"C Grade";
    }

    else if (num >= 70 && num < 80)
    {
        cout<<"B Grade";
    }

    else if (num >= 80 && num < 90)
    {
        cout<<"A Grade";
    }
}
```



```
}  
else if (num >= 90 && num <= 100)  
{  
    cout<<"A+ Grade";  
}  
}
```

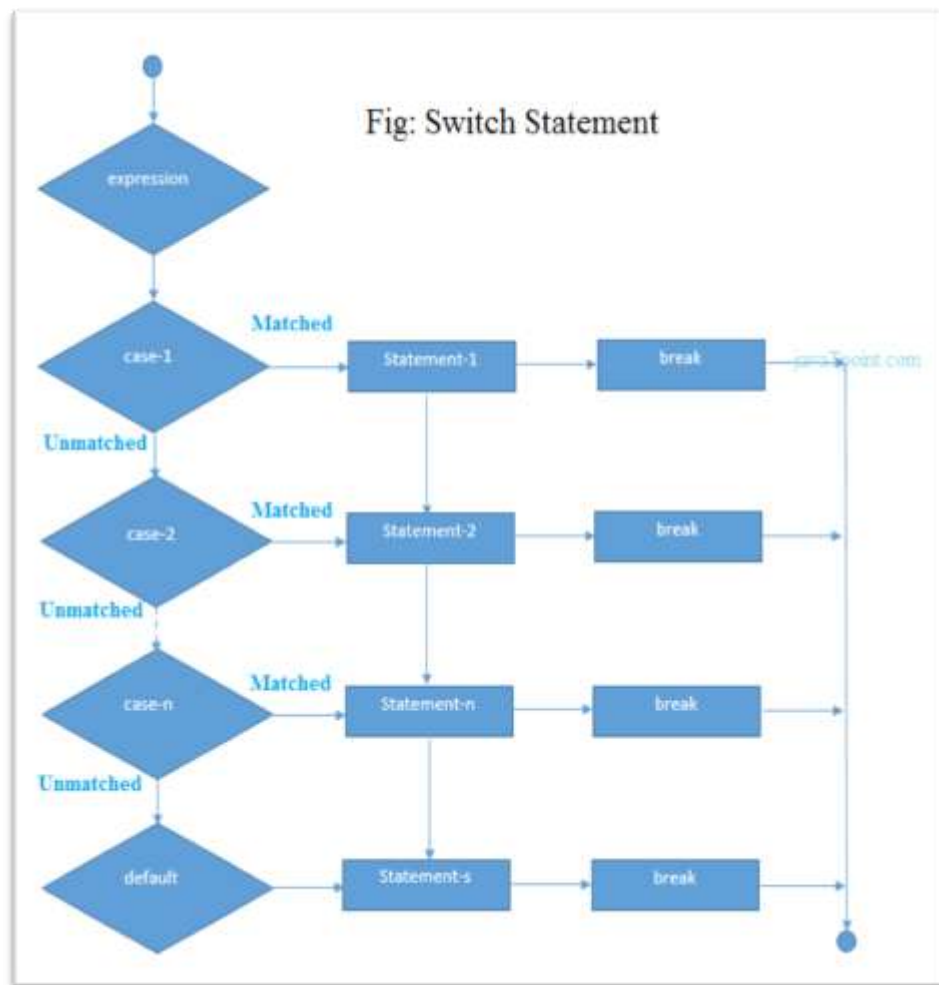
### Output

Enter a number to check grade:66  
C Grade

### Switch Statement

- The C++ switch statement executes one statement from multiple conditions.

```
switch(expression)  
{  
    case value1:  
        //code to be executed;  
        break;  
    case value2:  
        //code to be executed;  
        break;  
    .....  
    default:  
        //code to be executed if all cases are not matched;  
        break;  
}
```

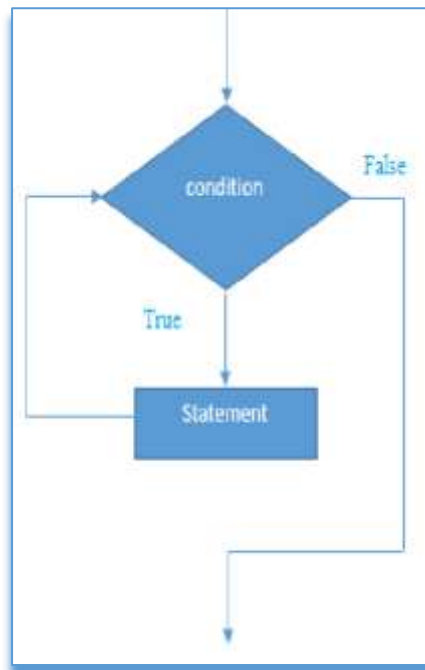


### Loop / Iterative / Repetitive Statements

- An iteration (or looping) is a sequence of one or more statements that are repeatedly executed until a condition is satisfied. These statements are also called as control flow statements.
- C++ supports three types of iteration statements;
  1. for statement
  2. while statement
  3. do-while statement
- In C++, while loop is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop than for loop.

```

while(condition){
    //code to be executed
}
  
```



```
#include <iostream>
using namespace std;
int main() {
    int i=1;
    while(i<=10)
    {
        cout<<i <<endl;
        i++;
    }
}
```

**Output:**

1  
2  
3  
4  
5  
6  
7



8

9

10

### Functions in C++

- The main function
- Function prototype
- Call by reference
- Return by reference
- Inline function
- Default arguments
- Functions overloading

**Recall that we have used a syntax similar to the following in developing C programs.**

```
void show();      /* Function declaration */

int main()
{
    ...
    show(); /* Function call */
    ...
    return (0);
}

void show()      /* Function definition */
{
    /* Function body */
}
```

- Dividing a program into functions is one of the major principles of top-down, structured programming.
- Another advantage of using functions is that it is possible to reduce the size of a program by calling and using them at different places in the program.
- When the function is called, control is transferred to the first statement in the function body.



- The other statements in the function body are then executed and control returns to the main program when the closing brace is encountered.
- C++ has added many new features to functions to make them more reliable and flexible.
- Like C++ operators, a C++ function can be overloaded to make it perform different tasks depending on the arguments passed to it.
- Most of these modifications are aimed at meeting the requirements of object-oriented facilities.

### THE Main Functions

- In C++, the main() returns a value of type int to the operating system.
- C++. therefore, explicitly defines main() as matching one of the following prototypes:

```
int main() ;
```

```
int main(int argc, char * argv(1);
```

- The functions that have a return value should use the return statement for termination. The main() function in C++ is, therefore, defined as follows:

```
int main()
```

```
{
```

```
.....
```

```
return 0;
```

```
.....
```

```
}
```

Since the return type of functions is int by default, the keyword int in the main() header is optional.

Most C++ compilers will generate an error or warning if there is no return statement.

Turbo C++ issues the warning.

### Function should return a value

and then proceeds to compile the program.

The explicit use of a return(0) statement will indicate that the program was successfully executed.

#### Function Elements

1. Function Prototype / Declaration
2. Function Definition
3. Function Call

## Function Prototyping

- The prototype describes the function interface to the compiler by giving details such as the number and type of arguments and the type of return values.
- With function prototyping, a template is always used when declaring and defining a function.
- When a function is called, the compiler uses the template to ensure that proper arguments are passed, and the return value is treated correctly.
- Any violation in matching the arguments or the return types will be caught by the compiler at the time of compilation itself.
- While C++ makes the prototyping essential, ANSI C makes it optional.
- Function prototype is a declaration statement in the calling program and is of the following form:

**type function-name (argument-list);**

- The argument-list contains the **types and names of arguments** that must be passed to the function.
- Example:

**float volume(int x, float y, float z);**

Note that each argument variable must be declared independently inside the parentheses. That is, a combined declaration like `float volume(int x, float y, z);` is illegal.

- In a function declaration, the names of the arguments are dummy variables and therefore, they are optional.
- That is, the form **`float volume(int, float, float);` is acceptable at the time of declaration.**
- In the function definition, **argument names are required because the arguments must be referenced inside the function.** Example:

```
float volume(int a,float b,float c) {
```

```
float v = a*b*c;
```

```
}
```

- The function `volume()` can be invoked in a program as follows:

```
float cubes = volume(b1,w1,h1); // Function call
```

- The variable `bl`, `w1`, and `h1` are known as the actual parameters which specify the dimensions of cube.
- Their types (which have been declared earlier) should match with the types declared in the prototype.

### Example

```
#include <iostream>

using namespace std;

int sum(int, int); //function declaration – step 1

int main(){

    int num1, num2, x;

    cout<<"Enter integer number1: ";

    cin>>num1;

    cout<<"Enter integer number2: ";

    cin>>num2;

    cout<<"Result: "<<sum(num1, num2); //function call step 3

    return 0;

}

int sum(int a, int b){ //function definition – step 2

    return a+b;

}
```

### Output:

```
Enter integer number1: 12
Enter integer number2: 34
Result: 46
```

### Call by Reference

- In traditional C, a function call passes arguments by value. The called function creates a new set of variables and copies the values of arguments into them.
- The function does not have access to the actual variables in the calling program and can only work on the copies of values.
- But, there may arise situations where we would like to change the values of variables in the calling program.
- For example, swap the values of two variables. If a function is used for swap, then it should be able to alter the values of variables in the calling function, which is not possible if the call-by-value method is used.

- Provision of the reference variables in C++ permits us to pass parameters to the functions by reference.
- When we pass arguments by reference, the 'formal' arguments in the called function become aliases to the 'actual' arguments in the calling function.
- This means that when the function is working with its own arguments, it is actually working on the original data.
- Consider the following function:

```
void swap(int &a, int &b) // a and b are reference variables
{
    int t = a; // Dynamic initialization
    a = b;
    b = t;
}
```

- Now, if m and n are two integer variables, then the function call  
    swap(m, n) ;
  - will exchange the values of m and n using their aliases (reference variables) a and b.

### Call by Reference

#### Example

```
#include <iostream>

void swap(int &, int &); // function prototype

using namespace std;

int main(){
    int m,n;
    cout <<"Enter number m";
    cin >> m;
    cout <<"Enter number n";
    cin >> n;
    cout <<"Before swap function m=" <<m << " n=" << n <<endl;
    swap(m,n); //function call
    cout <<"After swap function m=" <<m << " n=" << n <<endl;
    return 0;
}
```



```
}  
void swap(int &a, int &b) // a and b are reference variables  
{  
    int t = a; // Dynamic initialization  
    a = b;  
    b = t;  
}
```

**Output:**

```
Enter number m12  
Enter number n23  
Before swap function m=12 n=23  
After swap function m=23 n=12
```

**Return by Reference**

- A function can also return a reference. Consider the following function:

```
int & max (int &x, int &y)  
{  
    if  
    (x > y) return x;  
    else  
    return y;  
}
```

- Since the return type of max() is int &, the function returns reference to x or y (and not the values).
- Then a function call such as max(a, b) will return a reference to either a or b depending on their values.
- This means that this function call can appear on the left-hand side of an assignment statement. That is, the statement max(a,b) = -1; is legal and assigns -1 to a if it is larger, otherwise -1 to b.



## Return by Reference

### Example

```
#include <iostream>

using namespace std;

int &max (int &, int &);    //function prototype

int main ()
{
    int a, b;
    cout << "Enter number a";
    cin >> a;
    cout << "Enter number b";
    cin >> b;
    int &large = max (a, b);    //function call
    cout << "Largest number is" << large << endl;
    return 0;
}

int &max (int &x, int &y)
{
    if (x > y)
        return x;
    else
        return y;
}
```

### Output :

```
Enter number a 12
Enter number b 23
Largest number is 23
```

## Inline Functions

- However, every time a function is called, it takes a lot of extra time in executing a series of instructions for tasks such as jumping to the function, saving registers, pushing arguments into the stack, and returning to the calling function.
- When a function is small, a substantial percentage of execution time may be spent in such overheads.
- To eliminate the cost of calls to small functions, C++ proposes a new feature called inline function.
- An inline function is a function that is expanded inline when it is invoked.
- That is, the compiler replaces the function call with the corresponding function code
- The inline functions are defined as follows:

```
inline function-header  
{  
    function body  
}
```

### Example:

inline double cube(double a)

```
{  
    return (a*a*a) ;  
}
```

- The above inline function can be invoked by statements like  
c = cube(3.0);  
d = cube(2.5+1.5);
- On the execution of these statements, the values of c and d will be 27 and 64 respectively.
- If the arguments are expressions such as 2.5 + 1.5. the function passes the value of the expression, 4 in this case.
- All we need to do is to prefix the keyword inline to the function definition.
- All inline functions must be defined before they are called.
- Usually, the functions are made inline when they are small enough to be defined in one or two lines.
- Remember that the inline keyword merely sends a request, not a command, to the compiler.



- The compiler may ignore this request if the function definition is too long or too complicated and compile the function as a normal function.
- Some of the situations where inline expansion may not work are:
  - For functions returning values, if a loop, a switch, or a goto exists.
  - If functions contain static variables.
  - If inline functions are recursive.

### Inline Functions - Example

```
#include <iostream>

using namespace std;

inline float mul(float x, float y)
{
    return(x*y);
}

inline double div(double p, double q)
{
    return(p/q);
}

int main()
{
    float a = 12.345;
    float b = 9.82;
    cout << mul(a,b) << endl;
    cout << div(a,b) << endl;
    return 0;
}
```

Output:  
121.228  
1.25713



## Default Arguments

- C++ allows us to call a function without specifying all its arguments.
- In such cases, the function assigns a default value to the parameter which does not have a matching argument in the function call.
- Default values are specified when the function is declared.
- The compiler looks at the prototype to see how many arguments a function uses and alerts the program for possible default values.
- Here is an example of a prototype (i.e., function declaration) with default values:

```
float amount(float principal, int period, float rate=0.15);
```

- The above prototype declares a default value of 0.15 to the argument rate.
- A subsequent function call like

```
value = amount(5000,7); // one argument missing
```

- passes the value of 5000 to principal and 7 to period and then lets the function use default value of 0.15 for rate.
- The call

```
value = amount (5000,5,0.12) ; // no missing argument
```

- passes an explicit value of 0.12 to rate.

## DEFAULT ARGUMENTS

- One important point to note is that only the trailing arguments can have default values and therefore we must add defaults from right to left.
- We cannot provide a default value to a particular argument in the middle of an argument list.
- Some examples of function declaration with default values are:

```
int mul(int i, int j=5, int k=10);           // legal
```

```
int mul(int i=5, int j);                     // illegal
```

```
int mul(int i=0, int j, int k=10);          // illegal
```

```
int mul(int i=2, int j=5, int k=10);        // legal
```

C++ example to demonstrate use of default arguments.

We don't have to write 3 sum functions, only one function works by using default values for 3rd and 4th arguments.

## Default Arguments

### Example

```
#include<iostream>

using namespace std;

int sum(int x, int y, int z=0, int w=0);

int main()
{
    cout << sum(10, 15) << endl;
    cout << sum(10, 15, 25) << endl;
    cout << sum(10, 15, 25, 30) << endl;
    return 0;
}

int sum(int x, int y, int z, int w)
{
    return (x + y + z + w);
}
```

Output:  
25  
50  
80

## Function overloading

Function overloading is a feature in C++ where two or more functions can have the same name but different parameters.

Function overloading can be considered as an example of polymorphism feature in C++.

```
#include <iostream>

using namespace std;

void print(int i);
void print(double f);
void print(char const *c);

int main() {
    print(10);
    print(10.10);
    print("ten");
}
```

```
    return 0;
}
void print(int i) {
    cout << " int " << i << endl;
}
void print(double f) {
    cout << " float " << f << endl;
}
void print(char const *c) {
    cout << " char* " << c << endl;
}
```

**Output:**  
**int 10**  
**float 10.1**  
**char\* ten**